Accelerating Computer Vision by Using Model Compression Techniques

Rahul Boddeda, Revanth Reddy Avuthu, Vamshi Korra
Department of Computer Science and Engineering,
National Institute of Technology Andhra Pradesh, Tadepalligudem–534101, India

Abstract

The rapid growth of deep learning-based computer vision systems has produced models of remarkable capability, yet these models are often far too large and computationally expensive for deployment on resource-constrained devices such as smartphones, embedded systems, and edge hardware. This paper investigates three complementary model compression strategies—Knowledge Distillation (KD), Network Pruning, and Quantization-Aware Training (QAT)—with the aim of producing compact models that preserve accuracy while meaningfully reducing parameter counts and inference costs. Within Knowledge Distillation we explore offline distillation using cross-entropy and KL divergence losses, feature-mapping distillation with MSE-based intermediate supervision, Contrastive Knowledge Distillation (CKD) using InfoNCE loss, and Multi-Teacher Trajectory Matching. For Network Pruning we compare L2-norm filter removal, Taylor-criterion pruning, and our proposed Inference-Aware Filter Importance (IAFB) method, which computes importance from gradient-activation interactions evaluated over a calibration set with median aggregation. In the quantization domain we evaluate a 1-bit weight and 2-bit activation (1W2A) scheme alongside a mixed-precision QAT approach that keeps sensitive layers in full precision. Experiments on CIFAR-10, CIFAR-100, and ImageNette show that CKD delivers the highest distillation accuracy, IAFB achieves roughly 70% compression while actually improving accuracy over the unpruned baseline, and mixed-precision QAT produces strong compression at a cost of fewer than two percentage points of accuracy. Together these results show that structured, inference-aware compression can make powerful vision models viable on constrained hardware without meaningful loss of predictive quality.

Keywords: Knowledge Distillation, Network Pruning, Quantization-Aware Training, Model Compression, Edge Deployment

Abbreviations

- **DN**: DenseNet-BC-100
- **EB0**: EfficientNet-B0
- **MV2**: MobileNetV2
- **R18**: ResNet-18
- **R50**: ResNet-50
- **INette**: ImageNette
- **C-10**: CIFAR-10
- **C-100**: CIFAR-100

1 Introduction

Deep neural networks have become the foundation of modern computer vision, delivering state-of-the-art performance across a wide range of tasks. That performance improvement has come at a cost: large-scale models demand substantial memory, processing power, and energy, making them unsuitable for edge devices such as mobile phones and embedded systems.

Model compression has therefore emerged as an active and practically important research area. Its goal is to reduce the complexity of a trained network while preserving as much of its predictive ability as possible. Three families of techniques have attracted particular attention: Knowledge Distillation, which transfers knowledge from a large teacher to a smaller student; Network

Pruning, which removes redundant computational units; and Quantization, which reduces the numerical precision of weights and activations.

Treating the three families independently—rather than immediately combining them—allows a cleaner analysis of each technique's strengths, failure modes, and compression-accuracy trade-offs. This paper conducts exactly that kind of systematic study. We implement and benchmark multiple variants within each family and, for pruning, propose a new importance criterion we call IAFB that is specifically designed to reflect filter importance as it manifests during inference rather than during training. The findings offer practical guidance for selecting and combining compression strategies in real deployment scenarios. the source code is available at https://github.com/rahboddeda/Accelerating-Computer-Vision-using-Model-Compression-Techniques.git

1.1 Problem Statement

Modern convolutional networks for visual recognition carry an inherent tension between representational capacity and deployment feasibility. A model with tens of millions of parameters trained on a large benchmark will generalize well on that benchmark yet will exceed the memory and latency budgets of most edge targets by a wide margin. Simply training a smaller architecture from scratch tends to produce inferior accuracy because smaller models have less capacity to learn rich feature hierarchies.

The central problem is therefore how to compress a high-capacity vision model so that the resulting compact model (a) operates within practical resource constraints, (b) retains accuracy close to the original, and (c) generalizes across different benchmark distributions. The complication is that different compression techniques make different assumptions about where redundancy lives. Knowledge distillation assumes that soft probability distributions from a teacher encode generalizable knowledge. Pruning assumes that many filters contribute negligibly to the output. Quantization assumes that high-precision representations are wasteful. A further complication is that the relationship between compression and accuracy is not monotone: aggressive compression can cause accuracy collapse, while mild compression may yield negligible savings. Identifying the operating point where compression is both significant and accuracy-preserving is a non-trivial empirical question.

1.2 Objectives

The specific objectives are as follows. First, evaluate multiple KD variants—standard offline distillation, feature-level distillation, Contrastive KD, and trajectory matching—and characterize how each affects student accuracy under ResNet-50 supervision. Second, compare L2-norm pruning, Taylor-criterion pruning, and the proposed IAFB method on compression percentage, accuracy retention, and robustness to calibration data. Third, evaluate 1W2A and mixed-precision QAT in terms of accuracy degradation and efficiency gains. Fourth, synthesize findings to provide practical guidance on when each technique should be preferred. Finally, demonstrate that inference-aware filter importance scoring produces superior compression outcomes compared to classical criteria that do not account for inference-time behavior.

2 Literature Review

The theoretical foundation for Knowledge Distillation was established by Hinton et al. [5], whose key insight was that the probability distribution a well-trained large model assigns across all classes carries more information per training sample than a one-hot label. The non-trivial probability mass assigned to incorrect classes—termed dark knowledge—encodes relational information about class similarities and spawned a large research thread exploring richer forms of knowledge transfer.

Feature-level distillation was advanced by Romero et al. through FitNets [26], which introduced intermediate hint-layer supervision requiring a learned linear projection between teacher and student feature maps when their channel dimensions differ. Park et al. subsequently proposed Relational Knowledge Distillation, transferring structural relationships between samples rather than individual activation values. Contrastive principles were brought to distillation by Tian et al., who formulated KD as maximizing mutual information between teacher and student representations using a contrastive loss

built on the InfoNCE objective of van den Oord et al. Trajectory-based distillation, explored by Cazenavette et al. [2], showed that aligning intermediate parameter states during optimization provides a temporally distributed supervisory signal that complements output-only methods.

Early pruning work by LeCun et al. [20] used second-order derivative information to identify and remove unimportant weights. Han et al. [19] later demonstrated that large networks can tolerate the removal of upwards of 90% of connections when magnitude-based pruning is combined with retraining, though unstructured weight pruning often fails to translate to real inference speedups on standard hardware. Structured pruning, which removes entire filters or channels, directly reduces floating-point operations. Molchanov et al. [1] proposed a Taylor expansion criterion that estimates the loss change caused by removing each filter as the product of gradient and activation magnitude, consistently outperforming L1 and L2 norm approaches. Subsequent work explored batch-normalisation scaling factors, entropy-based criteria, and network architecture search as alternative ways to identify expendable filters.

Quantization research has progressed from post-training methods to QAT, which inserts fake quantization operations during training so the model learns to compensate for precision reduction. Jacob et al. [15] described a practical 8-bit QAT framework that has become standard in deployment pipelines. More aggressive schemes, including binary networks introduced by Courbariaux et al. [28] and XNOR-Net by Rastegari et al., showed that extreme compression is achievable with moderate accuracy cost on simpler datasets. Mixed-precision approaches that assign different bit-widths to different layers based on sensitivity analysis represent a practical middle ground between full precision and uniform low-bit quantization.

3 Methodology

3.1 Knowledge Distillation

Knowledge Distillation in this work follows an offline protocol: the teacher model (ResNet-50) is fully trained and its weights are frozen before student training begins. The student sees both hard ground-truth labels and the soft output distribution produced by the teacher at a controlled temperature T. Letting z_t and z_s denote teacher and student logits respectively, the soft targets are $\sigma(z_t/T)$ and $\sigma(z_s/T)$. The total distillation loss is:

$$\mathcal{L} = \alpha\,\mathcal{L}_{\text{CE}}(y, \sigma(z_s)) + (1-\alpha)\,T^2\,\text{KL}(\sigma(z_t/T) \| \sigma(z_s/T)) \tag{1}$$

where α balances the two terms and the T^2 factor compensates for gradient scaling. In all standard runs $\alpha = 0.5$ and $T = 4.0$.

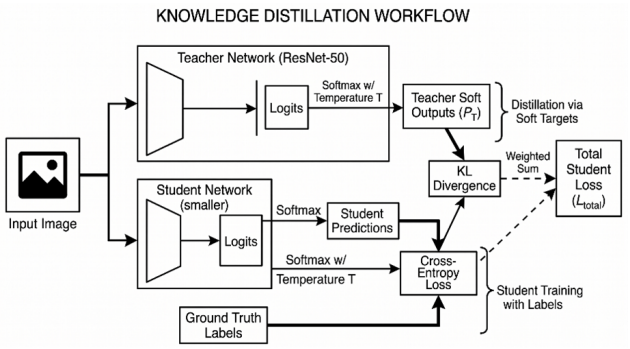

Figure 1: Basic Knowledge Distillation Workflow

Feature Mapping KD extends the loss with intermediate supervision. For each designated pair of teacher–student layers, an MSE loss is computed over spatially pooled feature representations. Because ResNet-50 and MobileNetV2 differ in channel dimensions, a learned 1×1 convolution adapter projects student features into the teacher space before comparison:

$$\mathcal{L}_{\text{FM}} = \mathcal{L}_{\text{KD}} + \beta\,\text{MSE}(f_t, A(f_s)) \tag{2}$$

When teacher and student architectures share similar inductive biases, as with ResNet-50 and ResNet-18, the feature mapping loss provides meaningful guidance. With heterogeneous architectures such as ResNet-50 and MobileNetV2, the adapter struggles to align fundamentally different feature distributions, leading to training instability.

Contrastive KD treats knowledge transfer as a representation alignment task. The InfoNCE loss pulls the student embedding of each sample toward the corresponding teacher embedding while pushing it away from embeddings of other samples in the batch:

$$\mathcal{L}_{\text{CKD}} = \mathcal{L}_{\text{KD}} + \gamma\,\mathcal{L}_{\text{InfoNCE}}(r_t, r_s) \tag{3}$$

This naturally encourages the student to preserve the relational geometry of the teacher's representation space, which is particularly useful on fine-grained datasets such as CIFAR-100 and ImageNette.

Adaptive Feature Consolidation (AFC) extends knowledge distillation by preserving important internal feature representations during training. Unlike standard KD, which treats all features equally, AFC assigns importance weights to feature channels based on their contribution to the loss and constrains changes in critical features. The feature preservation loss is defined as:

$$\mathcal{L}_{\text{AFC}} = \sum_{\ell,c} I_{\ell,c} \cdot \|Z'_{\ell,c} - Z_{\ell,c}\|^2 \tag{4}$$

where $I_{\ell,c}$ denotes feature importance and Z, Z' represent teacher and student features. The overall objective becomes:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{KD}} + \beta \mathcal{L}_{\text{AFC}}. \tag{5}$$

To address limitations of AFC, AFC++ introduces multi-layer feature alignment, attention-based importance estimation, and feature normalization for improved stability across architectures, along with a consistency regularization term:

$$\mathcal{L}_{\text{cons}} = \|f(x) - f(x+\delta)\|^2. \tag{6}$$

The final objective is:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha \mathcal{L}_{\text{KD}} + \beta \mathcal{L}_{\text{AFC}} + \gamma \mathcal{L}_{\text{cons}}, \tag{7}$$

resulting in more robust and effective knowledge transfer.

3.2 Network Pruning

All pruning experiments use structured filter pruning: entire convolutional filters are removed rather than individual weight connections, directly reducing FLOPs and producing a dense model that runs efficiently on standard hardware. The workflow is train baseline, rank filters by importance, remove the lowest-ranked fraction, fine-tune to recover accuracy, and repeat until the target compression is reached.

L2-Norm Pruning assigns importance by the L2 norm of each filter's weight tensor. It is purely weight-based, requires no data, but ignores how filters interact with the input distribution during inference.

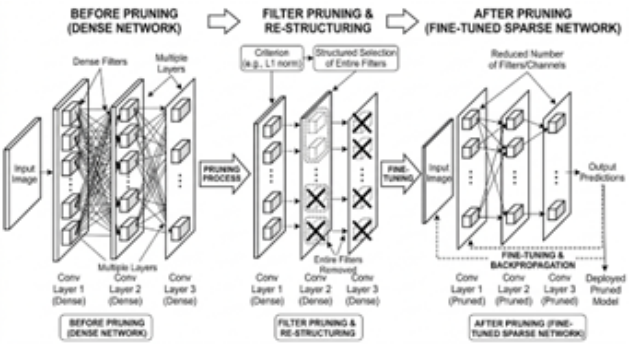

Figure 2: Basic Network Pruning Workflow

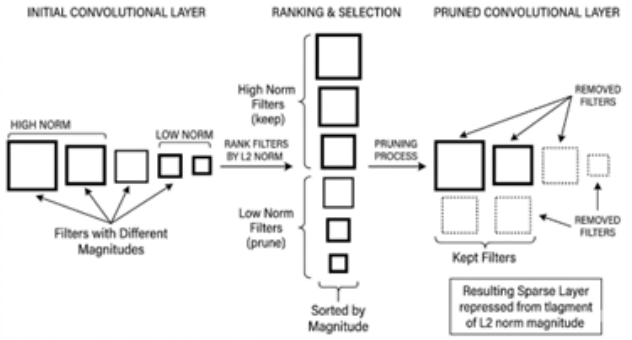

Figure 3: L2-Norm Based Pruning

Taylor Pruning estimates the change in training loss caused by zeroing each filter's output. The first-order importance of filter i is:

$$\Theta_i = |\nabla_{\mathbf{a}_i} \mathcal{L} \cdot \mathbf{a}_i| \tag{8}$$

This captures both loss sensitivity and filter activation, consistently outperforming L2 pruning but requiring gradient computation over training mini-batches.

Inference-Aware Filter Importance (IAFB) is the method proposed in this work. Its core motivation is that importance computed from training-time gradients can be influenced by optimization dynamics—gradient noise, batch-to-batch variation—that are irrelevant to inference behavior. IAFB computes importance over a dedicated calibration set of 2000 images drawn from the training distribution. Critically, it uses *median* rather than mean aggregation across calibration samples:

$$\Phi_i = \text{median}_{n \in \mathcal{C}} \left(|\nabla_{\mathbf{a}_i^{(n)}} \mathcal{L} \cdot \mathbf{a}_i^{(n)}| \right) \tag{9}$$

Mean aggregation is sensitive to outlier samples that produce anomalously large gradient-activation products, artificially inflating the score

of filters that respond strongly to a small subset of unusual inputs. Median aggregation is more robust to such outliers and provides an importance estimate that better reflects typical inference behavior. Table 1 shows a filter analysis of the first 10 layers of EfficientNet-B0, confirming that pruned filters are both weaker (lower L2 norm) and noisier (higher variance) than kept filters.

Table 1: Filter analysis on first 10 layers of EfficientNet-B0

Layer	Ch.	Prune	Var Ratio	Norm Ratio
features.0.0	32	6	1.518x	0.913x
features.2.0.block.3.0	24	4	1.060x	0.988x
features.3.0.block.3.0	40	8	1.096x	0.955x
features.4.0.block.2.fc1	10	2	0.981x	1.008x
features.4.1.block.3.0	80	16	1.003x	0.998x
features.5.0.block.2.fc1	20	4	1.001x	0.998x
features.5.1.block.3.0	112	22	0.993x	1.004x
features.6.0.block.2.fc1	28	5	1.033x	0.984x
features.6.1.block.3.0	192	38	1.006x	0.997x
features.6.3.block.2.fc1	48	9	1.004x	0.998x

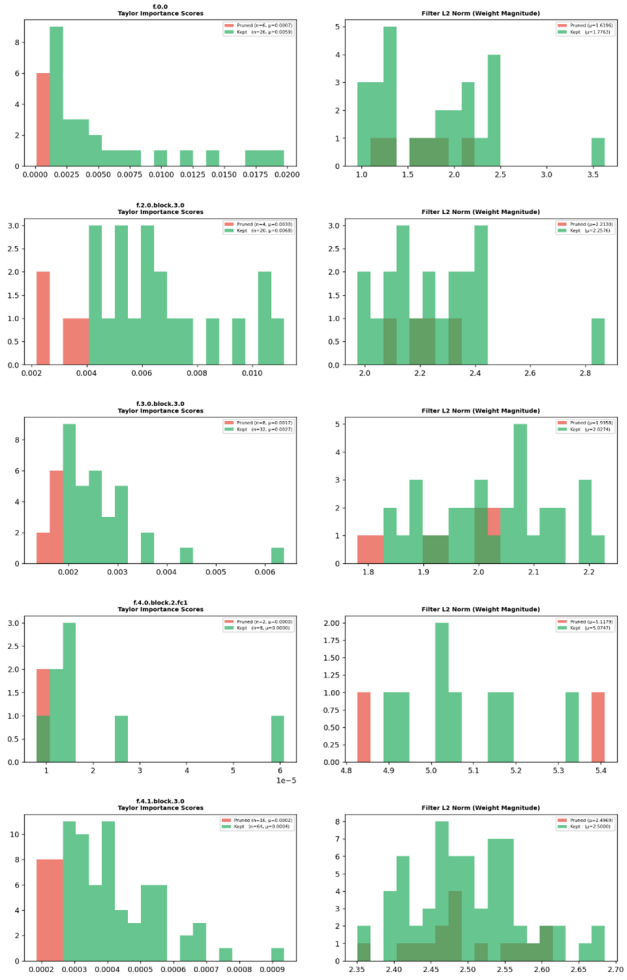

Figure 4: Taylor Importance of Filters and L1-norm

3.3 Quantization

Unlike post-training quantization, QAT inserts simulated quantization operations into the forward pass during training so the optimizer can compensate for precision loss through gradient updates.

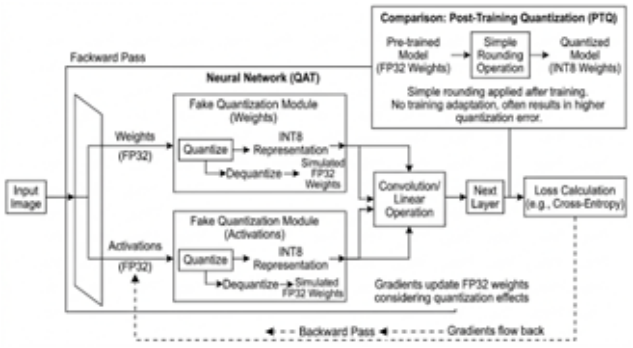

Figure 5: Basic Quantization Aware Training Workflow

1W2A binarizes each weight to ± 1—reducing weight storage by 32× relative to FP32—and quantizes activations to 4 discrete levels using a 2-bit fixed-point representation. Straight-through estimators allow gradients to flow through the discontinuous binarization functions. First and last

layers are kept in full precision because they interface with raw pixel inputs and class probabilities where precision loss is most damaging.

Mixed-Precision QAT assigns different precisions to different layers based on sensitivity to quantization error. Sensitive layers—identified by measuring the per-layer validation accuracy drop when each layer is independently quantized while all others remain in full precision—are retained in FP32; the rest are quantized to INT8. A warm-up phase of several full-precision epochs precedes quantization activation to prevent the sudden accuracy collapse that occurs when quantization is applied abruptly. In practice, layers causing more than 0.3 percentage points of accuracy loss when individually quantized are designated sensitive.

Modified Mixed Precision partitions the network into INT8 early layers and FP32 deeper layers to preserve accuracy-critical representations while still achieving computational efficiency. Early layers, which capture low-level features, are quantized to INT8 for speed and memory savings, whereas deeper layers remain in full precision to maintain semantic fidelity. Compared to standard mixed precision, this approach improves training stability through the use of exponential moving average (EMA) updates and more robust optimization strategies, reducing quantization noise and ensuring smoother convergence with higher final accuracy.

ADQ + Mixed Precision extends mixed quantization-aware training by introducing a weight-alignment loss that minimizes the discrepancy between full-precision weights and their quantized counterparts. For each layer, weights are quantized via scaling and clamping, and an additional L2 regularization term enforces closeness between the original and quantized values. This mechanism effectively guides weights toward discrete quantization levels during training, reducing the mismatch introduced during quantization-aware training. As a result, the transition to INT8 becomes smoother, leading to improved stability and reduced accuracy degradation.

4 Experimental Setup

All experiments run on PyTorch with a single 16 GB GPU. The KD teacher is ResNet-50 pretrained on ImageNet and fine-tuned on each target dataset. Student architectures are MobileNetV2 and ResNet-18. Three datasets are used: CIFAR-10 (60,000 32×32 images, 10 classes), CIFAR-100 (the same scale across 100 classes), and ImageNette (a 10-class ImageNet subset with 9,469 training and 3,925 validation images at higher resolution). KD students train for 100 epochs using SGD with momentum 0.9, initial learning rate 0.01, and cosine annealing. Temperature is fixed at 4.0. Pruning iteratively removes the bottom 10% of filters by the chosen criterion, fine-tunes for 10 epochs per step, and uses a 2000-image calibration set for IAFB. Quantization experiments run for 80 epochs (1W2A) or a 10-epoch warm-up followed by 70 quantization-aware epochs (mixed-precision). All accuracy figures are top-1 on held-out test sets, averaged over three random seeds.

5 Results and Analysis

5.1 Knowledge Distillation

Standard offline KD improves student accuracy over hard-label training on all three datasets. On CIFAR-10, ResNet-18 with standard KD reaches 93.8% versus 92.5% without any teacher supervision. Feature Mapping KD is architecturally sensitive: it gives marginal gains for ResNet-18 on CIFAR-10 (94.1%) but degrades below standard KD for MobileNetV2 on CIFAR-100 and ImageNette because the learned feature adapter cannot align the fundamentally different feature distributions of ResNet-50 and inverted residual blocks. CKD achieves the highest accuracy across all datasets and both student architectures—ResNet-18 under CKD reaches 76.3% on CIFAR-100 versus 74.7% for standard KD. On ImageNette, the ResNet-18 student trained with CKD reaches 93.2%, marginally above the teacher's 92.9%, attributable to the regularization effect of contrastive training. Trajectory matching is effective for ResNet-18 but shows limited value for MobileNetV2, where the architectural gap makes parameter-space alignment semantically weaker. Table 2 summarizes representative results.

5.2 Network Pruning

Standard network pruning of DenseNet-BC-100 at 51% compression shows mixed behavior: accuracy improves by 1.49% on CIFAR-100 but drops by 0.30% on CIFAR-10. Filter pruning at ∼50% compression is more stable, producing gains of

Table 2: Knowledge distillation results

Method	Teacher	Student	Dataset	ΔAcc
Standard KD	R50	MV2	C-100	−8.67%
Standard KD	R50	MV2	C-10	−0.78%
Feature Map	R50	MV2	C-100	−9.60%
CKD	R50	MV2	C-100	+0.27%
CKD	R50	R18	C-100	−7.80%
AFC	R50	MV2	C-10	−8.32%
AFC	R50	MV2	C-100	−13.01%
AFC++	R50	MV2	C-10	−5.56%
AFC++	R50	MV2	C-100	−12.66%

+0.14% (CIFAR-100), +0.56% (CIFAR-10), and +0.76% (ImageNette). Taylor importance pruning at ∼43% compression degrades CIFAR-100 accuracy by 1.62%, indicating that gradient information alone does not fully capture global filter importance under high compression.

IAFB consistently outperforms all baselines across architectures and datasets, even at ∼70% compression. ResNet-50 on CIFAR-10 gains 2.14 percentage points; on CIFAR-100 it gains 2.75 points. MobileNetV2 gains 3.74% on CIFAR-100 and 3.04% on CIFAR-10 at ∼48% compression. EfficientNet-B0 sees gains of 3.99% and 7.76% on CIFAR-100 and CIFAR-10 respectively at only ∼34% compression. These accuracy improvements are consistent with the interpretation that IAFB removes filters whose presence, through weight interaction and gradient interference, slightly harms generalization. Table 3 reports selected results.

Table 3: pruning results (selected)

Method	Model	Dataset	Prune	ΔAcc
Network Pruning	DN	C-100	51%	+1.49%
Filter	DN	C-10	50%	+0.56%
Taylor	DN	C-100	43%	−1.62%
IAFB	R50	C-10	70%	+2.14%
IAFB	R50	C-100	70%	+2.75%
IAFB	MV2	C-100	48%	+3.74%
IAFB	EB0	C-10	34%	+7.76%

5.3 Quantization

1W2A introduces noticeable degradation: 3.49% on CIFAR-10, 2.87% on CIFAR-100, and 4.08% on ImageNette. Combining KD with 1W2A substantially mitigates this—using a ResNet-50 teacher and ResNet-18 student on CIFAR-10 reduces the drop to just 0.70%. Mixed-precision QAT is considerably more accurate: ResNet-50 drops only 2.12% on CIFAR-10, while ResNet-18 loses just 0.52%. AutoHybrid QAT on ResNet-18 achieves effectively no degradation on CIFAR-10 (−0.02%). Table 4 summarizes these results.

Table 4: Quantization results

Method	Model	Dataset	ΔAcc
1W2A	R50	C-10	−3.49%
1W2A	R50	C-100	−2.87%
1W2A	R50	INette	−4.08%
KD + 1W2A	R50→R18	C-10	−0.70%
Mix.Prec.	R50	C-10	−2.12%
Mix.Prec.	R18	C-10	−0.52%
Modified Mix. Prec.	R18	C-10	−0.04%
ADQ + mixed QAT	R18	C-10	−1.12%
ADQ + mixed QAT	R18	C-100	+3.68%

6 Discussion

CKD's consistent advantage over other distillation variants stems from the fact that contrastive training captures relational geometry across the full embedding space rather than only pointwise class probabilities. The regularization introduced by InfoNCE loss also appears to prevent students from overfitting teacher idiosyncrasies, which explains occasional student outperformance on test sets. Feature Mapping KD's sensitivity to architecture mismatch is a practical concern: the adapter introduces a secondary optimization objective whose gradient updates can conflict with the student's primary task, leading to slow convergence or poor local minima. Practitioners should apply feature-level distillation cautiously when teacher and student architectures are heterogeneous.

IAFB's advantage over Taylor pruning rests on two factors: a dedicated calibration set rather than random training mini-batches, and median rather than mean aggregation. Mean aggregation is distorted by rare samples with anomalously

large gradient-activation products; median aggregation is inherently more robust. The calibration set is small enough—2000 images—that the additional cost is negligible compared to the fine-tuning budget. A practical ceiling on compression exists for all pruning methods: beyond a certain removal fraction, the remaining filters are insufficient to represent the necessary feature transformations and accuracy collapses regardless of criterion quality.

Mixed-precision QAT is the most practically useful quantization scheme evaluated here. The warm-up phase is a critical implementation detail: applying quantization abruptly can trap the model in suboptimal weight configurations that are difficult to escape with extended training. The sensitivity threshold for designating layers as FP32-retained is a hyperparameter that must be tuned per architecture; too conservative a threshold leaves compression gains on the table, while too aggressive a threshold sacrifices accuracy.

7 Conclusion

This paper has presented a systematic evaluation of Knowledge Distillation, Network Pruning, and Quantization-Aware Training for compressing computer vision models. Contrastive KD emerged as the most effective distillation variant, achieving state-of-the-art accuracy among distillation methods and occasionally enabling students to outperform their teachers. Feature Mapping KD should be applied with caution when architectures differ substantially. The proposed IAFB pruning method outperformed both L2-norm and Taylor baselines at $\sim$70% compression by combining an inference-representative calibration set with median aggregation. Mixed-precision QAT delivered the best quantization accuracy, staying within two percentage points of full-precision baselines across all configurations.

These findings demonstrate that model compression is a design space with multiple operating points. Practitioners with inference latency as the primary constraint will benefit most from structured pruning; those with memory constraints will find quantization most directly useful; those seeking the best accuracy from a small student will benefit from CKD. When all three constraints apply simultaneously, the techniques can be stacked: a student trained via CKD can subsequently be pruned with IAFB and then quantized with mixed-precision QAT, accumulating compression benefits across all three dimensions.

Acknowledgements

The authors sincerely thank Mr. K. Subrahmanya Kousik for his guidance and support throughout this project.

References

[1] P. Molchanov, A. Mallya, S. Tyree, I. Frosio, and J. Kautz, "Importance estimation for neural network pruning," in *Proc. CVPR*, 2019.

[2] G. Cazenavette, T. Wang, A. Torralba, A. A. Efros, and J.-Y. Zhu, "Dataset distillation by matching training trajectories," in *Proc. CVPR*, 2022.

[3] M. Kirtas *et al.*, "Mixed-precision quantization-aware training for photonic neural networks," *Applied Physics B*, vol. 130, no. 60, 2024.

[4] D. Qin *et al.*, "Efficient medical image segmentation based on knowledge distillation," 2021.

[5] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[6] G. Aguilar *et al.*, "Knowledge distillation from internal representations," 2020.

[7] Z. Li, H. Li, and L. Meng, "Model compression for deep neural networks: A survey," 2023.

[8] T. Choudhary, V. Mishra, A. Goswami, and J. Sarangapani, "A comprehensive survey on model compression and acceleration," 2020.

[9] M. Gupta and P. Agrawal, "Compression of deep learning models for text: A survey," 2022.

[10] W. Wang *et al.*, "Model compression and efficient inference for large language models: A survey," 2024.

[11] H. Wang *et al.*, "Recent advances on neural network pruning at initialization," 2021.

[12] Z. Wang, C. Li, and X. Wang, "Convolutional neural network pruning with structural redundancy reduction," 2021.

[13] R. Yu *et al.*, "NISP: Pruning networks using neuron importance score propagation," 2018.

[14] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv preprint arXiv:1806.08342*, 2018.

[15] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. CVPR*, 2018.

[16] K. Wang *et al.*, "HAQ: Hardware-aware automated quantization with mixed precision," 2019.

[17] C. Li and C. Shi, "Constrained optimization based low-rank approximation of deep neural networks," 2018.

[18] Z. Liu *et al.*, "A ConvNet for the 2020s," 2022.

[19] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural networks," in *Adv. NeurIPS*, 2015.

[20] Y. LeCun, J. S. Denker, and S. A. Solla, "Optimal brain damage," in *Adv. NeurIPS*, 1990.

[21] B. Hassibi and D. G. Stork, "Second order derivatives for network pruning: Optimal brain surgeon," in *Adv. NeurIPS*, 1993.

[22] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *Proc. ICLR*, 2019.

[23] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient ConvNets," in *Proc. ICLR*, 2017.

[24] Z. Luo, M. Wu, and S. Lin, "ThiNet: A filter level pruning method for deep neural network compression," in *Proc. ICCV*, 2017.

[25] W. Wen, C. Wu, Y. Wang, Y. Chen, and H. Li, "Learning structured sparsity in deep neural networks," in *Adv. NeurIPS*, 2016.

[26] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for thin deep nets," in *Proc. ICLR*, 2015.

[27] F. Tung and G. Mori, "Similarity-preserving knowledge distillation," in *Proc. ICCV*, 2019.

[28] M. Courbariaux, Y. Bengio, and J.-P. David, "BinaryConnect: Training deep neural networks with binary weights," in *Adv. NeurIPS*, 2015.

[29] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks," in *Adv. NeurIPS*, 2016.

[30] Y. Gong, L. Liu, M. Yang, and L. Bourdev, "Compressing deep convolutional networks using vector quantization," *arXiv preprint arXiv:1412.6115*, 2014.